Neurocomputing 558 (2023) 126701

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Cluster-aware attentive convolutional recurrent network for multivariate time-series forecasting

Simeng Bai [a], Qi Zhang [c,d], Hui He [b], Liang Hu [c,d], Shoujin Wang [e], Zhendong Niu [a,*]

[a] School of Computer Science and Technology, Beijing Institute of Technology, Beijing, 100081, China
[b] School of Medical Technology, Beijing Institute of Technology, Beijing, 100081, China
[c] Tongji University, Shanghai, 200092, China
[d] DeepBlue Academy of Sciences, Shanghai, 200336, China
[e] Data Science Lab, University of Technology Sydney, Sydney, Australia

ARTICLE INFO

Communicated by F.G. Guimaraes

Keywords:
Multivariate time series
Forecasting
Inter-series dependencies
Cluster-aware attention mechanism

ABSTRACT

Multivariate time-series (MTS) forecasting plays a crucial role in various real-world applications, but the complex dependencies between time-series variables (i.e., inter-series dependencies) make this task extremely challenging. While most existing studies focus on modeling intra-series (temporal) dependencies by capturing long- and short-term patterns, they fail to explore and exploit the inter-series dependencies to enhance MTS forecasting. In this paper, we propose a Cluster-aware Attentive Convolutional Recurrent Network (CACRN) to capture both inter-series and intra-series dependencies in MTS data. Specifically, CACRN first introduces a cluster-aware variable representation module that separates irrelevant variables and captures the interaction between relevant variables to learn cluster-aware variable representations. Then, CACRN feeds these representations into parallel convolutional recurrent neural networks (CRNNs) to capture the short- and long-term temporal dependencies in a cluster-wise manner. Next, a cluster-aware attention mechanism is introduced to attend to temporal information in each cluster and co-attend all cluster information jointly to capture intra-cluster and inter-cluster dependencies for the downstream forecasting task. Our extensive experiments on six real-world datasets demonstrate that CACRN is effective and outperforms representative and state-of-the-art baselines. Our proposed method is suitable for a wide range of real-world data collections, especially those with clear dependencies of variables.

1. Introduction

Multivariate time series (MTS) forecasting has played an essential role and benefited our daily life in a wide range of practical fields including traffic flow monitoring [1], financial analysis [2], air pollution analysis [3], energy price and demand planning [4,5]. In particular, with the rapid progress of advanced neural networks, recent deep learning-based models, e.g. LSTM-based models [6,7], Transformer-based models [8] and GNN-based models [9,10], have achieved outstanding performance in various applications [11].

Noteworthy, many existing MTS forecasting models pay more emphasis on long- and short-term temporal patterns, while overlooking the benefits of considering variable dependencies (inter-series dependencies) in MTS. For instance, multiple financial factors, such as prices, volumes, and volatility, are interdependent and co-evolve in market analysis. Another example is as shown in Fig. 1(a), which is sampled from a common MTS dataset, Air Quality dataset.[1] Obviously, we observe strong dependencies are evident between certain variables (e.g. C6H6 (GT), PT08.S2 (NMHC), PT08.S1 (CO), and PT08.S5 (03)), while weaker dependencies exist between other variables (e.g. C6H6 (GT) and RH). This is understandable since dependent variables are more likely to influence each other significantly, while independent variables may not have a mutual influence and may even introduce noise or bias into the forecasting task. Fig. 1(b) demonstrates that the forecasting performance improves when all the variables in the input data are related, but it decreases when irrelevant variables are incorporated into the input data. Therefore, it highlights the importance of precisely capturing the inter-series dependencies in MTS data to achieve accurate multivariate time-series forecasting.

Recently, advanced deep learning-based models have recently begun to consider inter-series dependencies in addition to intra-series dependencies for MTS forecasting. Representative approaches including graph neural networks (GNNs) [12] and self-attention mechanisms [10]

* Corresponding author.
E-mail address: zniu@bit.edu.cn (Z. Niu).

[1] https://archive.ics.uci.edu/ml/datasets/Air+Quality.

https://doi.org/10.1016/j.neucom.2023.126701
Received 4 April 2023; Received in revised form 22 June 2023; Accepted 13 August 2023
Available online 16 August 2023
0925-2312/© 2023 Elsevier B.V. All rights reserved.

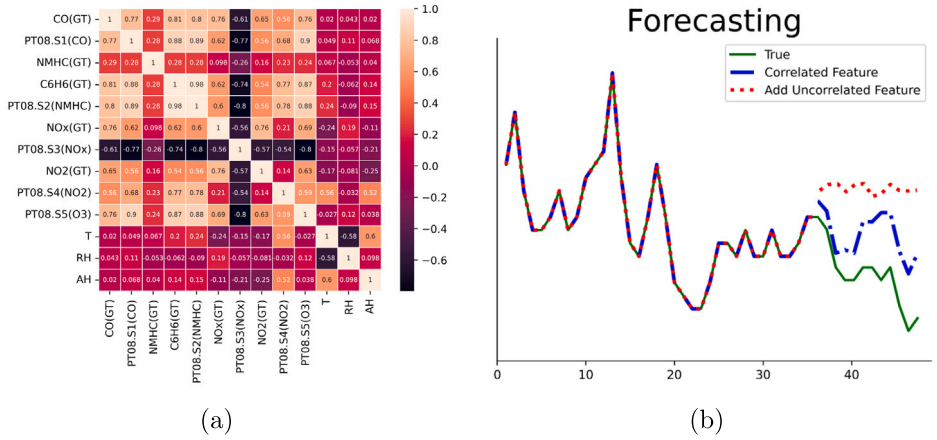


Fig. 1. Analysis of the Air Quality dataset. (a): Correlation visualization of time-series variables by heat-map, (b) Forecasting results from two kinds of extra inputs additive to the target time series: the blue line denotes predictive results obtained from the closely related variables to the ground truth time series, while the red line is obtained from uncorrelated variables.

have been employed to capture these dependencies. However, these methods either rely heavily on a predefined graph structure to indicate variable correlations or implicitly and inefficiently learn different correlations to weigh all variables. Furthermore, they fail to explicitly segregate time-series variables to separate irrelevant variables and gather dependent variables, which may hinder their ability to effectively reduce the potential effects of noise and bias embedded in irrelevant variables and improve forecasting performance.

To this end, we first present a suitable time-series clustering method inspired by recent studies on time-series clustering [13–15] to separate uncorrelated time-series variables. Then we feed the correlated time-series variables into the prediction module in parallel and capture the dependencies between different clusters using an attention mechanism at last. Thus, we can model each time series independently in a cluster-aware manner, and avoid representing MTS with a single-strand vector. Furthermore, it is essential to effectively model the dependencies between multiple time-series variables in each cluster (i.e., inter-series dependencies). The latest studies [16,17] have shown a promising approach to capturing the inter-series dependencies in a pairwise fashion. This method unitizes variable embeddings to calculate pairwise dependencies via decomposing the correlation matrix of variables into learnable embedding parameters [16]. This gives us ideas for capturing the interaction between multiple variables in the same cluster.

In view of the above discussion, we propose a Cluster-aware Attentive Convolutional Recurrent Network (CACRN)that simultaneously integrates intra-series and inter-series dependencies in a cluster-wise manner. Firstly, we adopt a *Variable Fuzzy Clustering* method to segregate the time-series variables according to their correlations. Secondly, we introduce a *Variable Interaction Representation* module to capture the interactions between variables within each cluster and learn a fusion representation of cluster-wise time-series variables. Next, the cluster-aware representations of all clusters are fed into different *Convolutional Recurrent Neural Networks* in a one-to-one manner. The CRNN is comprised of a convolutional component for capturing short-term temporal dependencies and a recurrent component for capturing long-term temporal dependencies. Finally, we consider a *Cluster-aware Attention Mechanism* to extract the intra-cluster and inter-cluster dependencies via integrating the outputs of the CRNNs with attention to each other. The major contributions have been listed below:

- We present a Cluster-aware Attentive Convolutional Recurrent Network (CACRN) for the task of MTS forecasting, which captures intra-series and inter-series dependencies in a cluster-aware manner.
- We introduce a cluster-aware variable representation module that models inter-series dependencies via segregating irrelevant time-series variables over time and capturing interactions between variables that are correlated with each other.
- We consider a cluster-aware attention mechanism to integrate the outputs of multiple convolutional recurrent neural networks in order to capture the inter-cluster and intra-cluster dependencies.
- Experiments are conducted on six real-world datasets, and the findings reveal that the proposed CACRN has superior effectiveness compared to the state-of-the-art baseline.

The remainder of this article is structured as follows. The related research in the area of time-series forecasting is covered in Section 2. The issue and definitions for the time-series forecasting task are presented in Section 3. The suggested model CACRN is described in depth in Section 4. The specifics of the experiment and the analysis of the findings are covered in Section 5. Additionally, Section 6 provides a summary of this paper.

2. Related work

This section summarizes the relevant work in the field of time-series forecasting. First, we summarize the classical statistical methods. After understanding the shortcomings of classical statistical methods, deep learning methods are discussed. Among them, we analyzed typical methods that can mine intra-series (temporal) dependencies. However, these methods ignore the complex dependencies between variables. In this regard, we analyzed the methods of capturing inter-series dependencies. After finding their problems, we draw the solution in a cluster-aware manner for CACRN. Finally, the methods of clustering for time series prediction are sorted out, and the differences and innovations of CACRN are analyzed.

2.1. Classical statistical models

In the past, the task of time-series forecasting mainly relied on statistical models, including the autoregressive integrated moving average (ARIMA) [18], ARIMAX [19], GARCH [20]. One of the most successful methods for univariate time series is the ARIMA model due to the well-known Box-Jenkins methodology, which is a desirably accurate algorithm for analyzing and forecasting time series. However, these univariate time-series forecasting models are not applicable to multivariate time series. Based on the aforementioned statistical models, the vector autoregression (VAR) method was introduced for multivariate time-series forecasting subsequently [21–23]. VAR models can be regarded as a broadening of AR processes by including additional time series

such that they can handle multivariate forecasting tasks by incorporating the field of multivariate time-series analysis. However, both AR- and VAR-based models may fall short in capturing the non-linear correlations and hardly extend to large-scale multivariate datasets.

2.2. Deep learning models

Recurrent neural networks (RNN) and convolutional neural networks (CNN), which are good at capturing temporal (intra-series) dependencies, are two examples of deep neural networks that have lately gained popularity for capturing non-linear features. For the objective of time-series forecasting, popular models like long short-term memory (LSTM) [24–26] and gate recurrent unit (GRU) [27] have been used extensively. A representative model, the long- and short-term time-series network (LSTNet) [6], employs CNN and LSTM for the first time to capture short-term patterns and memorize long-term patterns, respectively. The requirement for the skip length makes LSTNet inappropriate for time series with variable period lengths, which is one of its primary drawbacks. To address the issue, temporal pattern attention (TPA-LSTM) [7] was proposed by replacing the recurrent-skip layer with a temporal attention layer and obtaining better performance. The above methods effectively capture the temporal patterns to a certain extent, but they still perform poorly in capturing long-term patterns. Recently, Transformer-based models [8,28,29] have been introduced into time-series analysis to obtain the temporal correlations. Among them, Informer [8] facilitates solving the problem of long-sequence time-series forecasting via self-attention.

Although these models have good performance in multivariate time-series forecasting, they are not competent in solving the complex dependencies between multi-variables. Recent studies adopt graph neural networks to learn the relationship between variables in the forecasting task of multivariate time series. For example, MTGNN [9] captures the relationship between variables by introducing graph neural networks, and spectral temporal graph neural network (StemGNN) [10] joint obtains two aspects inter-series and temporal dependencies respectively. However, these methods fail to explicitly segregate time-series variables to avoid noises from unrelated variables and facilitate capturing the dependencies among related variables. CATN [26] solves this problem by constructing the tree structure. Its hierarchical structure can separate unrelated variables from each other by dividing them into different layers. As a preliminary work, CATN has achieved good results. However, we found that it would depend on the determination of the number of layers. From this point on, we use the non-hyperparameter-dependent and more efficient Cluster-aware Variable Representation module to realize the mining of inter-series dependencies.

2.3. Clustering based models

Recent studies pay increasing attention to multivariate time-series clustering that learns to cluster time-series variables into different groups [30]. Existing models can be generally divided into two categories: *raw-data-based* and *feature-based* models [31,32]. Raw-data-based models design the distance function to capture series characteristics, suffering from noises and being insensitive to temporal information [33,34]. Feature-based models extract time-series representations and learn to cluster based on the extracted features, which effectively alleviates the influence of noise and outliers [32,35,36], or solving data scarcity problems based on similar features and trend analysis [37]. In feature-based models, the main challenge is extracting or learning effective time-series representations from sequences. To this end, the latest deep-learn models learn feature representation and clustering simultaneously, which is capable to learn nonlinear high-level features. In our studies, we attempt to jointly learn nonlinear time-series representations and cluster time-series variables, which facilitates to capture of complex inter-series dependencies varying over time.

3. Preliminaries

Assume a collection of time series $\mathrm{x}^i = \{x_1^i, x_2^i, \ldots, x_{o_t}^i\}$ being the observed values of the ith variable at time stamp o_t. Given the fixed input observation window w, the input is denoted as $X = \{\mathrm{x}^1, \mathrm{x}^2, \ldots, \mathrm{x}^n | \mathrm{x}^n \in \mathbb{R}^w\}$, where n means the variate dimension. And the output is the corresponding sequence $Y = \{\mathrm{y}^1, \mathrm{y}^2, \ldots, \mathrm{y}^n | \mathrm{y}^n \in \mathbb{R}^h\}$, where h means the length of forecasting and it may differ from w. Under a rolling forecasting fashion, we adopt a window-sized input $X_{o_t-w+1}^{o_t}$ sliding over the T-length multivariate time series. We consider the task of forecasting the h-length future values for each variable at the time point o_t, i.e., $\{x_{o_t+1}^i, x_{o_t+2}^i, \ldots, x_{o_t+h}^i\}$. Accordingly, the multivariate time-series forecasting for all variables is accordingly formalized as follows:

$$\begin{aligned} &\hat{Y} = \{\hat{y}_{o_t+1}, \hat{y}_{o_t+2}, \ldots, \hat{y}_{o_t+h}\} \\ &= F(X_{o_t-w+1}^{o_t}) = F(\mathrm{x}_{o_t-w+1}, \mathrm{x}_{o_t-w+2}, \ldots, \mathrm{x}_{o_t}) \end{aligned} \tag{1}$$

where x_{o_t}, $\hat{y}_{o_t+1}$ denote the multivariate observations at time step o_t and its corresponding prediction sequence at the next time point. And F denotes the forecasting model, e.g., our proposed CACRN. Therefore, obtaining a forecast of future values in a rolling forecasting fashion based on historical values is the purpose of multivariate time-series forecasting.

The proposed CACRN considers the temporal dependencies with each time-series variable. And the time-series input is also fed into the cluster-aware variable representation module to separate the irrelevant variables from each other and then capture the interaction between the relevant variables. In this way, we not only consider the intra-series dependencies but also capture inter-series dependencies during multivariate time-series forecasting.

4. Methodology

In the description of this section, we introduce the framework of our proposed model Cluster-aware Attentive Convolutional Recurrent Network (CACRN), which is composed of three components: Cluster-aware Variable Representation module (CVR), Convolutional Recurrent Neural Network (CRNN), and Cluster-aware Attention Mechanism (CAM). The framework of our CACRN is illustrated in Fig. 2. Through the time series $X = \{\mathrm{x}^1, \mathrm{x}^2, \ldots, \mathrm{x}^n | \mathrm{x}^n \in \mathbb{R}^w\}$ in an observation window w, the calculation process of model CACRN is further described as follows. Firstly, CACRN obtains the dependencies relationship between variables through two components in the CVR module. The first component uses the Variable Fuzzy Clustering method to separate variables into different clusters $\{C_1 = \{\mathrm{x}^a, \ldots, \mathrm{x}^b\}, C_2 = \{\mathrm{x}^i, \ldots, \mathrm{x}^j\}, \ldots, C_k = \{\mathrm{x}^p, \ldots, \mathrm{x}^q\}\}$. The second component catches the correlations and gets the embedding of variables between the variables in the same cluster $C_i \in \{C_1, C_2, \ldots, C_k\}$, where k is the number of clusters. Secondly, we integrate the temporal information with the cluster-wise time-series representation and feed the mixed representation to CRNN to calculate the non-linear temporal dependencies in parallel. Finally, CACRN designs CAM to capture the inter-series dependencies between variables and adopts the fully-connected layers for generating predictions.

4.1. Cluster-aware Variable Representation module

In this subsection, an innovative module is presented to capture the dependencies between different variables, which contains two components: Variable Fuzzy Clustering and Variable Interaction Representation.

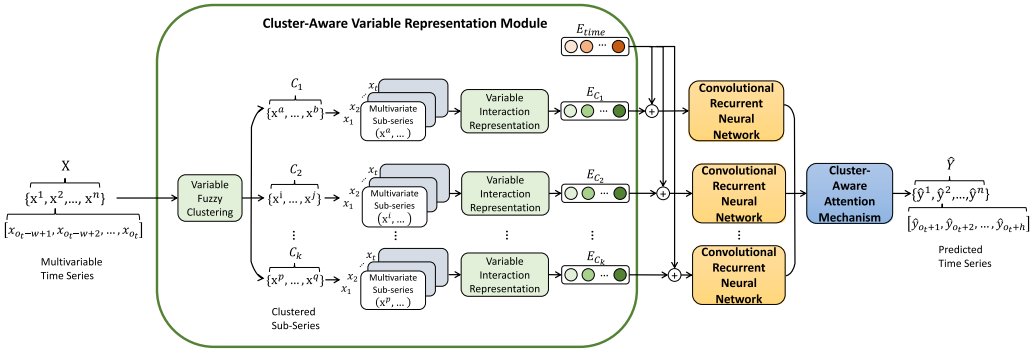


Fig. 2. The architecture of the Cluster-aware Attentive Convolutional Recurrent Network (CACRN).

4.1.1. Variable Fuzzy Clustering

In previous studies, K-Means clustering has often been used to cluster time-series variables [38]. However, this algorithm heavily depends on the parameter k to complete the clustering process, which can be challenging to set appropriately in practical application scenarios, leading to inaccurate clustering results. Additionally, K-Means is a hard clustering algorithm that assumes a variable can only belong to one class, which may not hold in the case of time-series data, where the relationship between variables changes over time. Similar issues are encountered with other hard clustering algorithms, such as hierarchical clustering [26].

To address these limitations, we introduce a Variable Fuzzy Clustering method inspired by the Fuzzy C-Means algorithm [36]. This method aims to separate uncorrelated variables from each other based on observed time-series data, and it allows for the soft assignment of variables to clusters, meaning that a variable can belong to multiple clusters with different degrees of membership. This approach is better suited for time-series data as it captures the dynamic nature of the relationship between variables over time without depending on the parameter k heavily.

Given a multivariate time-series data matrix $\mathcal{X} = \{\mathrm{x}^1, \mathrm{x}^2, \dots, \mathrm{x}^n | \mathrm{x}^n \in \mathbb{R}^w\}$, VFC aims to dividing $\mathcal{X}$ into k clusters, $[C_1, C_2, \dots, C_k]$ according to the Euclidean distance between variables. The objective function of the VFC must be minimized with the help of its necessary conditions, which are expressed as:

$$\mathbf{J}(\mathrm{x}^i, C_j) = \sum_{i=1}^{n}\sum_{j=1}^{k} \mu_{ij}^m \| \mathrm{x}^i - C_j \|^2; 1 < m < \infty \tag{2}$$

where μ_{ij}^m means the membership value of the x^i in jth cluster, m is the fuzziness weighting exponent and C_j is the cluster centroid.

In the process of minimization, the membership degree matrix U and the center matrix V need to be updated alternately until convergence is achieved.

$$\mu_{ij}^m = \sum_{l=1}^{k} \left(\frac{\| \mathrm{x}^i - C_j \|}{\| \mathrm{x}^i - C_l \|} \right)^{\frac{2}{m-1}-1} \tag{3}$$

$$C_j = \frac{\sum_{i=1}^{n} \mu_{ij}^m \mathrm{x}^i}{\sum_{i=1}^{n} \mu_{ij}^m} \tag{4}$$

where $\sum_{i=1}^{n} \mu_{ij} = 1$ and $\mu_{ij} \in [0, 1]; i = 1, 2, \dots, n$.

The Variable Fuzzy Clustering (VFC) method utilizes a membership degree matrix U to indicate the probability that each variable belongs to each cluster. This allows for the possibility of variables changing the cluster they belong to by altering their membership degree over time. Unlike the K-Means algorithm, VFC does not heavily rely on a predefined value of k to complete the clustering process. In other words, VFC does not require all variables to be allocated to k clusters. Instead, VFC determines the number of clusters based on the membership degree matrix U at the end of the iteration. This approach enables a more reasonable division of each variable in the time-series data into different clusters, facilitating the separation of variables based on their relationships over time.

4.1.2. Variable Interaction Representation

Unlike traditional models that consider each variable independently, our approach focuses on the relationship between variables within the same cluster. By separating uncorrelated variables from each other using VFC, we can better capture the correlations between variables in the same cluster. To achieve this, we introduce a Variable Interaction Representation (VIR) module that allows us to model and capture these correlations between the variables within the same cluster. Inspired by [16], VIR obtains the intra-cluster variable correlations by calculating the pairwise interactions between variables given a real-valued variable vector $\mathrm{x} \in \mathbb{R}^{w \times n_k}$, where w indicates the observation window and n_k indicates how many variables are in the kth cluster.

$$VIR = w_0 + \sum_{i=1}^{n} w_i \mathrm{x}_i + \sum_{i=1}^{n}\sum_{j=i+1}^{n} \langle v_i, v_j \rangle \mathrm{x}_i \circ \mathrm{x}_j \tag{5}$$

where w_0 means the overall bias, w_i means the weight of the ith variable, and $\langle v_i, v_j \rangle$ means the weight of the pair-wise interaction of variables x_i and x_j and $\circ$ denotes the element-wise product.

VIR introduces pairwise variable correlations and weights in addition to obtaining interactions between variables. We consider the correlated variables $\mathrm{x} \in \mathbb{R}^{w \times n_k}$ as input of VIR and get the embedding of variables in each cluster, denoted as $E_{C_i} \in \mathbb{R}^{w \times n_k \times d}$, where d means the embedding dimension. Accordingly, the embedding of the correlated variables of each cluster can be obtained $E_C = \{E_{C_1}, E_{C_2}, \dots, E_{C_k}\} \in \mathbb{R}^{w \times d}$. Through VIR, we can capture the interaction of variables in the same cluster, and form a cluster-wise representation of intra-cluster dependencies.

To capture the temporal information in time series data, we not only consider the variable embeddings but also the temporal factors. To achieve this, we transform the timestamps into hierarchical time factors (e.g., year, month, week, and day) and consider the time period (A.M. or P.M.) to obtain a more comprehensive representation of time. We then extract n hierarchical timestamps as $T_n \in \mathbb{R}^{w \times n}$ and use VIR to obtain the final time embedding as $E_{time} \in \mathbb{R}^{w \times d}$. Similarly, in addition to the variable embeddings, the CRNN also receives the time embedding to capture the temporal information. By incorporating the time factor into the model, we can better capture the temporal dynamics in the time-series data and improve the performance of the model.

The combination of VFC and VIR forms the Cluster-aware Variable Representation (CVR) module, which allows us to capture the dependencies between different variables by dividing them into clusters and achieving cluster-wise representation through intra-cluster pairwise variable interactions. We then use the cluster-wise representations as inputs for the Convolutional Recurrent Neural Network (CRNN) in parallel to obtain cluster-wise outputs. The CVR module enables us to model and capture the complex relationships between variables in the time-series data and incorporate this information into the CRNN for improved performance.

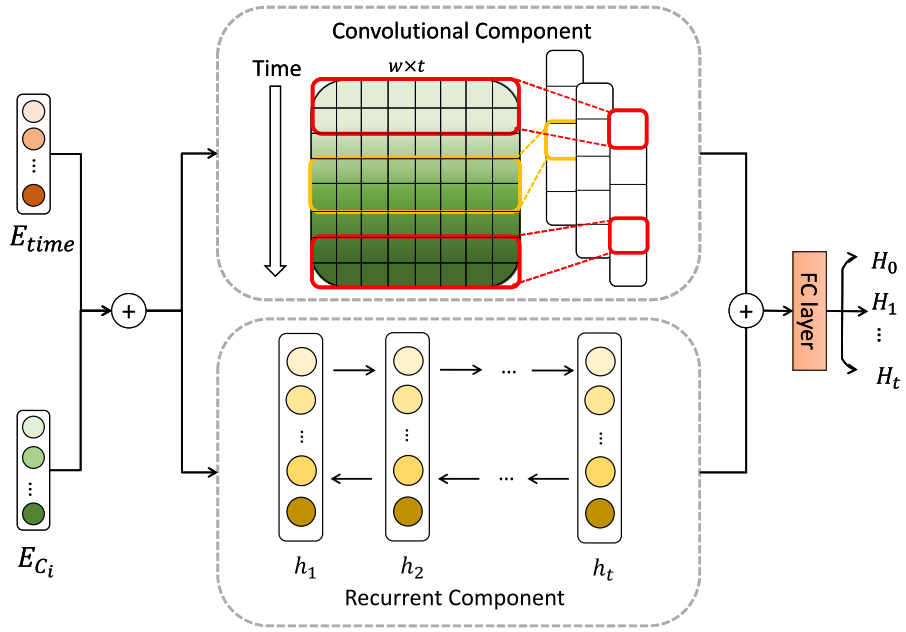


Fig. 3. The architecture of the Convolutional Recurrent Neural Network.

4.2. Convolutional Recurrent Neural Network

We carry out the forecasting task in the Convolutional Recurrent Neural Network in each cluster in parallel by integrating the convolutional and recurrent networks to mine two types of pattern information: short-term and long-term information, respectively. Fig. 3 depicts the CRNN's basic architecture.

4.2.1. Convolutional component

We feed the embeddings of the variables, i.e., $E_C = \{E_{C_1}, E_{C_2}, \dots, E_{C_k}\}$, and temporal embeddings E_{time} into the convolutional component where local temporal patterns can be captured by local temporal convolution in the time dimension. The convolutional component's first layer is a convolutional layer without pooling, inspired by TPA-LSTM [7]. Specifically, k filters with width w_c and length l_c have been used and we adopt $l_c = d_c$ here. We combine the output of variable representations E_{C_i} and E_{time} as the input E_C of all filters slide through:

$$o_C = \frac{1}{k}\sum_{i=1}^{k} RELU(W_i * E_C + b_i) \tag{6}$$

where o_C means the output of the convolutional component and $*$ represents the convolution operation. By applying a zero-padding operation, we zero-fill the left side of the input matrix E_C and thus preserve the original input size. In this way, we obtain the d-length of each vector o_C and accordingly the output with a size of $w \times k$.

4.2.2. Recurrent component

Because of its exceptional long and short-term memory, long short-term memory (LSTM) [39] is frequently utilized in the time-series forecasting task. LSTM is able to learn long-term information through the gate mechanism. It has three gates that control what information is stored, including input, forget, and output gates.

Given $\{x_{o_t-w+1}, x_{o_t-w+2}, \dots, x_{o_t}\}$ as a sequence of information from the observation time point o_t, where $x_t \in \mathbb{R}^n$ and t denotes the time step within the window-sized input, LSTM cells have a recurrent function:

$$h_t, c_t = LSTM(h_{t-1}, c_{t-1}, x_{o_t}) \tag{7}$$

The detailed calculation of LSTM is shown as follows, where x_t is the input value for this layer at time point t, σ means the function of the sigmoid, and $\odot$ means the element-level product operation.

$$\begin{aligned} r_t &= \sigma(x_t W_{xr} + h_{t-1} W_{hr} + b_r) \\ u_t &= \sigma(x_t W_{ur} + h_{t-1} W_{hu} + b_r), \\ c_t &= RELU(x_t W_{xc} + r_t \odot (h_{t-1} W_{hc}) + b_c), \\ h_t &= (1 - u_t) \odot h_{t-1} + u_t \odot c_t \end{aligned} \tag{8}$$

As shown in the above formulas, the LSTM unit computes a hidden state h_t and a memory state s_t, which are the encodings of the input time-series data up until t time steps during one observation window w. However, the forward context of the same sequence of data cannot be learned by LSTM; it can only be learned by time series in the past. Therefore, we use the bidirectional LSTM, i.e., Bi-LSTM to comprehensively take into account the data before and after the current step, which combines two normal LSTMs in order to capture dependencies both in forward and backward directions:

$$h_t = \overrightarrow{h_t} \circ \overleftarrow{h_t} \tag{9}$$

As a result, we may acquire two hidden state sequences, the forward sequence $(\overrightarrow{h_t^1}, \overrightarrow{h_t^2}, \dots, \overrightarrow{h_t^k})$ being one and the backward sequence $(\overleftarrow{h_t^1}, \overleftarrow{h_t^2}, \dots, \overleftarrow{h_t^k})$ being the other. After getting these two hidden state sequences, we denote the hidden states of Bi-LSTMs as $(h_1, h_2, \dots, h_k)$ after the concatenating operation $\circ$, where k means the number of clusters and parallels of Bi-LSTMs. We take the hidden states of paralleled Bi-LSTMs combined with o_C into the combination layer as its inputs.

4.3. Cluster-aware Attention Mechanism

Through the Convolutional Recurrent Neural Network, we obtain k outputs for each cluster and regard the cluster-wise outputs as the inputs of the Cluster-aware Attention Mechanism. The Cluster-aware Attention Mechanism has two attention components: an intra-cluster attention component and an inter-cluster attention component. The framework of the Cluster-Aware Attention Mechanism is shown in Fig. 4.

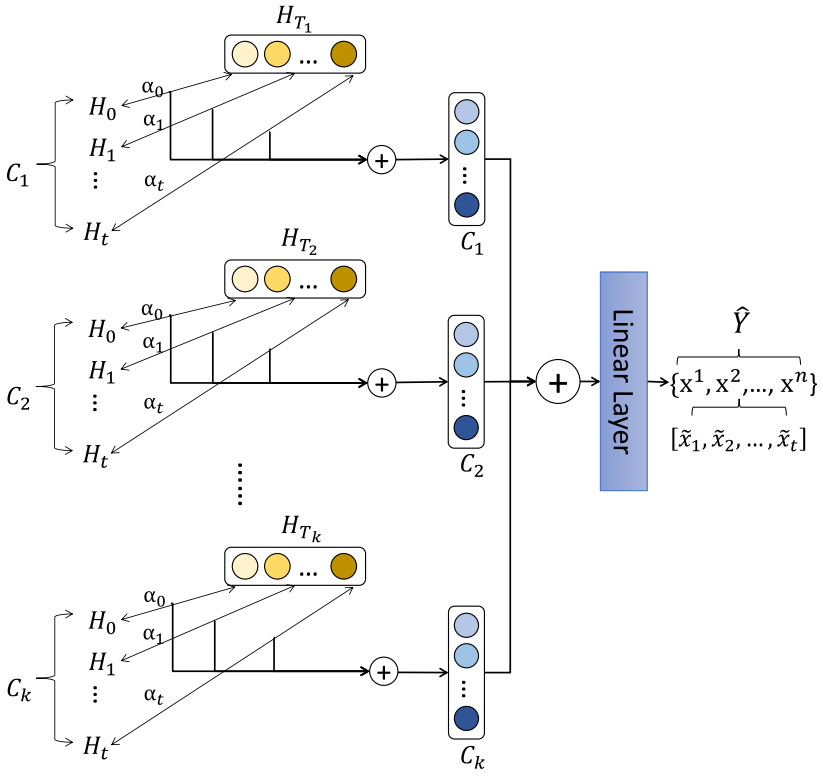


Fig. 4. The architecture of the Cluster-Aware Attention Mechanism.

4.3.1. Intra-cluster attention component

In order to better capture the intra-cluster temporal dependencies, we explore an intra-cluster attention mechanism that obtains the attentively weighted summation of hidden states at each time stamp of the time period data. Given the current time stamp t, the attention weights $\alpha_t \in \mathbb{R}^w$ are formulated as follows:

$$\alpha_t = SScore(H_t^i, h_{t-1}^i) \tag{10}$$

where $H_t^i = [h_{t-w+1}^i, \dots, h_{t-1}^i]$ means the matrix stacking the hidden states of CRNN in cluster C_i and $SScore$ is the similarity methods, such as cosine and dot product.

Through a concatenation operation, the hidden states can be updated as $[H_t\alpha_t; h_{t-1}^i]$ and the intra-cluster attention component's outputs can be obtained as follows:

$$C_i = W[H_t\alpha_t; h_{t-1}^i] + b \tag{11}$$

4.3.2. Inter-cluster attention component

After obtaining the attentive representations for each cluster in parallel, we explore an inter-cluster attention mechanism that takes into account the inter-cluster dependencies. It means the various contributions of historical data from variables in different clusters to the final time-series forecasting results. Specifically, given a set of results obtained through the networks $\{C_1, C_2, \dots, C_k\}$, where k is the clustering number. The following is how the inter-cluster attention mechanism is described:

$$O = \sum_{i=1}^{k} \alpha_i C_i = \alpha_1 C_1 + \alpha_2 C_2 + \cdots + \alpha_k C_k \tag{12}$$

$$E_i = \tanh(Attn(O, C_i)), \tag{13}$$

$$\widetilde{\alpha_i} = \upsilon E_i, \tag{14}$$

$$\alpha_i = softmax(\widetilde{\alpha_i}) \tag{15}$$

where α_i is the correlation between O and C_i, and the function $Attn$ computes the results via the self-attention method [40] through the learnable parameters w_K and w_Q.

$$Attn(O, C_i) = (w_k \cdot C_i)^T (w_Q \cdot O) \tag{16}$$

By employing the inter-cluster attention component, we can get the different weights of different clusters and combine these weighted outputs into one final representation for forecasting. To sum up, these two attention components together form the Cluster-aware Attention Mechanism, which obtains the intra-cluster and inter-cluster dependencies respectively.

4.4. Learning objective

The final prediction of CACRN maps the final representation to the original vector space through fully-connected layers as follows:

$$\hat{Y} = RELU(\alpha[C_1; C_2; \cdots; C_k] + b) \tag{17}$$

where $[C_1; C_2; \cdots; C_k]$ is the concatenation of the output vector of each cluster and $\hat{Y}$ denotes the final prediction of h-steps.

During the training procedure, the Mean Squared Error(MSE) was adopted to compute and optimize the loss for multi-step forecasting together.

$$\mathcal{L}_{CACRN} = \frac{1}{h} \sum_{i=1}^{h} (Y[:, \tau] - \hat{Y}[:, \tau])^2 \tag{18}$$

where h is the horizon of forecasting and $\tau = o_t + 1 : o_t + h$. Each neural model is trained simultaneously using the Adam optimizer [41]. Through joint training of each module in CACRN, the parameters in the model are continuously updated to reflect the different contributions in the attention mechanism and obtain the optimal value of prediction. Additionally, the Variable Fuzzy Clustering module is independent learning. It continuously updates the partition results by incorporating the data of the next time period, and eventually obtains the optimal partition results. The subsequent learning of other modules is then carried out based on the optimal partition results obtained by VFC. By continually updating and optimizing the partition results, we can achieve more accurate and robust predictions of time-series data.

5. Experiment

In this section, we introduced six real-world datasets and state-of-the-art baselines adopted in empirical evaluations. Then, we outline our experimental procedures and give further analysis based on the experimental results. As for the influence of parameters and the contribution of various components in CACRN, we perform a parameter analysis and ablation study respectively. Finally, we investigate the effect of the inter-series dependencies to demonstrate the significance of considering the dependencies between variables in multivariate time-series forecasting tasks.

5.1. Dataset and metrics

To analyze the prediction accuracy of our proposed CACRN model, we validate it and compare it with baseline models on six datasets. These datasets are commonly used in multivariate time-series forecasting research [6,9,26,42], and cover various fields such as transportation, electricity, and the environment. To obtain robust comparison results, we selected these six representative datasets with varying data dimensions. By evaluating the performance of our model and comparing it with existing methods on a diverse range of datasets, we can demonstrate the effectiveness and generalizability of our proposed CACRN model. The statistics of these real datasets are provided in Table 1, where L denotes the time-series length, D denotes the number of variables, and S denotes the time-series sampling rate. The corresponding details are introduced as follows:

Traffic[2]: This dataset, which details the occupancy rate of various car lanes on highways in the San Francisco Bay Area, was made available by the California Department of Transportation. The measurements of dimension 963 were taken every 10 min.

Electricity[3]: The data includes 321 clients' electricity use in kWh, which was monitored every 15 min from 2012 to 2014.

Wiki^{small}[4]: Daily online traffic on around 115,084 Wikipedia articles. Due to our poor computing power, we reduce the dimensions of the original dataset to 5000 as a new dataset, namely Wiki^{small}.

PeMSD7(M)[5]: Caltrans Performance Measurement System (PeMS) collects data in real-time from almost 40,000 individual detectors spanning the freeway system throughout all of California's major urban centers.

Solar-Energy[6]: The Solar Power Data is made up of 5-min solar power from 137 photovoltaic facilities in Alabama over the course of a year (2006).

Air Quality[7]: The dataset contains a total of 9358 h of average response examples from March 2004 to February 2005. These responses are from five metal oxide chemical sensor arrays embedded in the air-quality chemical multi-sensor equipment.

Following the partition proportion adopted in most studies [6,9], we divide all datasets into training set (60%), verification set (20%) and test set (20%). We use two conventional evaluation metrics that are often used for the task of multivariate time-series prediction [8,9,26,42], including $RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(Y_t^i - \hat{Y}_t^i)^2}$, and $MAE = \frac{1}{N}\sum_{i=1}^{N}|Y_t^i - \hat{Y}_t^i|$, where $Y_t^i, \hat{Y}_t^i$ are ground true values and model forecasting results.

[2] https://archive.ics.uci.edu/ml/datasets/PEMS-SF.
[3] https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014.
[4] https://www.kaggle.com/c/web-traffic-time-series-forecasting/data.
[5] https://dot.ca.gov/programs/traffic-operations/mpr/pems-source.
[6] https://www.nrel.gov/grid/solar-power-data.html.
[7] https://archive.ics.uci.edu/ml/datasets/Air+Quality.

Table 1
The statistics of all datasets.

Dataset	L	D	S
Traffic	10 560	963	10 min
Electricity	26 136	370	1 h
Wiki^{small}	803	5000	1 day
PeMSD7	12 672	228	5 min
Solar	52 560	137	5 min
Air Quality	9358	13	1 h

5.2. Baselines

On multivariate time-series datasets, we compare our proposed model CACRN to the following methods:

VAR [43]: Christopher Sims proposed the use of VAR to create multivariate time-series prediction models based on the statistical characteristics of the data.

DARNN [44]: The authors proposed two attention mechanisms, an input attention mechanism, and a temporal attention mechanism, for extracting relevant drive sequences and selecting encoder hidden states.

LSTNet [6]: In order to identify short-term patterns of local dependence between variables and discover long-term patterns of trends in series data, the authors merged CNN with RNN to create the model.

TPA-LSTM [7]: The authors used filters to extract temporal patterns and propose attention mechanisms for multivariate temporal prediction.

MTGNN [9]: The authors proposed graph learning modules to extract the monomial relationships between variables and represent the correlations in spatial and temporal.

DeepGLO [42]: The authors implemented a multivariate time-series prediction task by combining a global matrix decomposition model and a local time network.

Informer [8]: The authors proposed a self-attentive mechanism based on the encoder–decoder structure for the prediction of long sequence time series.

TS2Vec [45]: The authors constructed a universal framework at an arbitrary semantic level by performing a contextual representation for each timestamp.

CATN [26]: The authors proposed a cross-focused tree-aware network model that recognizes both intra-series temporal trends and inter-series relationships through feature engineering, which is the pre-model of the proposed CACRN.

In the baseline as mentioned above, we take VAR as the representative of classical statistical models. Other various models include the state-of-the-art methods that depend on RNN, CNN, LSTM, and transformer. We adopt these methods to compare the effectiveness of different deep learning methods. Among them, TS2Vec has a novel idea to solve the problem of time-series prediction. It performs contrastive learning in a hierarchical way to get the universal Representation. MTGNN and CATN both consider the relationships between variables. MTGNN and CATN both consider the relationships between variables, with the graph learning module of MTGNN able to automatically extract the one-way relationship between variables, while CATN constructs a tree structure through feature engineering to mine inter- and intra-sequence dependencies. These two methods learn inter-series dependencies like our proposed CACRN, so we compared their performance to validate the effectiveness of our proposed method.

5.3. Experimental settings

The selected baseline models and our model CACRN were implemented in Python and trained on two servers equipped with a Graphics Processing Unit (GPU) of NVIDIA Geforce RTX 3080 and 10 GB memory. The CACRN model was trained using the Adam optimizer with an initial learning rate of 2×10^{-4} and a dropout rate of 0.5, which

Table 2
Metrics results (RMSE, MAE) of all methods on the datasets of Traffic, Electricity, Wiki^{small}. In each case, the best performance is displayed in **boldface**. (1) each row contains the two-metric findings of a given approach; (2) each column evaluates the outputs of all approaches on a specific dataset with different horizons.

Models	Metrics	Traffic				Electricity				Wiki^{small}			
		3	6	9	12	3	6	9	12	3	6	9	12
VAR	RMSE	0.037	0.068	0.098	0.194	0.241	0.256	0.277	0.279	0.052	0.054	0.056	0.057
	MAE	0.025	0.048	0.064	0.102	0.081	0.087	0.096	0.111	0.019	0.021	0.023	0.025
DARNN	RMSE	0.022	0.022	0.023	0.024	0.341	0.404	0.414	0.404	0.676	0.708	0.715	0.721
	MAE	0.051	0.055	0.051	0.054	0.486	0.565	0.581	0.565	0.267	0.278	0.283	0.284
LSTNet	RMSE	0.085	0.048	0.051	0.052	0.235	0.245	0.263	0.271	0.074	0.074	0.078	0.079
	MAE	0.019	0.021	0.022	0.024	0.076	0.084	0.087	0.084	0.031	0.032	0.033	0.034
TPA-LSTM	RMSE	0.031	0.032	0.041	0.051	0.249	0.269	0.276	0.272	0.029	0.262	0.721	0.392
	MAE	0.019	0.021	0.022	0.024	0.062	0.068	0.074	0.074	0.023	0.184	0.109	0.151
MTGNN	RMSE	0.052	0.079	0.059	0.061	0.295	0.317	0.323	0.313	0.053	0.054	0.055	0.056
	MAE	0.024	0.047	0.029	0.031	0.065	0.082	0.087	0.077	0.021	0.020	0.021	0.021
DeepGLO	RMSE	0.035	0.037	0.033	0.045	0.027	0.028	0.052	0.093	0.032	0.032	0.027	0.028
	MAE	0.018	0.022	0.019	0.029	0.021	0.012	0.012	0.012	0.017	0.017	0.016	0.015
Informer	RMSE	0.023	0.028	0.029	0.031	**0.013**	0.014	0.015	0.015	0.023	0.025	0.024	0.023
	MAE	0.015	0.014	0.016	0.015	0.013	0.014	0.014	0.014	0.028	0.023	0.024	0.024
TS2Vec	RMSE	0.027	0.029	0.031	0.031	0.027	0.029	0.029	0.031	0.037	0.039	0.041	0.043
	MAE	0.016	0.017	0.018	0.018	0.021	0.023	0.023	0.025	0.021	0.024	0.026	0.027
CATN	RMSE	0.031	0.031	0.032	0.031	0.260	0.261	0.262	0.265	0.027	0.027	0.029	0.028
	MAE	0.018	0.018	0.019	0.018	0.061	0.061	0.061	0.063	0.022	0.022	0.024	0.024
CACRN (ours)	RMSE	**0.022**	**0.021**	**0.026**	**0.024**	0.014	**0.014**	**0.015**	**0.013**	**0.022**	**0.023**	**0.023**	**0.022**
	MAE	**0.014**	**0.013**	**0.015**	**0.015**	**0.011**	**0.011**	**0.012**	**0.011**	**0.010**	**0.011**	**0.011**	**0.010**

was selected based on previous research findings [46]. We repeated each experiment three times and reported the average results. During training, we used a batch size of 16 and stopped the training after 50 epochs or when the validation set performance stopped improving.

During the forecasting task, we let the parameter forecasting horizon h over $\{3, 6, 9, 12\}$, which represents that the horizons were placed between 15 and 60 min above the PeMSD7, 3 to 12 days above the Wiki^{small} dataset, 3 to 12 h above the Electricity and Air Quality datasets, and 30 to 120 min above the Traffic and Solar-Energy datasets. We tune hyperparameters by grid searching the size of observation window w over $\{12, 24, 36\}$, the length of embedding dimension d over $\{32, 64, 128\}$ and the number of variable fuzzy clustering $k \in [2, 20]$. Generally, we set $w = 36$ and $d = 128$ as the optimal hyperparameters, which will be explained through the subsequent parameter analysis. The hyperparameter k for the variable fuzzy clustering is determined by the VFC method, which calculates the appropriate number of clusters based on the membership degree matrix. On the basis of this value, we further take different k values for validation and select the value that achieves the highest accuracy on the validation set. The optimal k value varies across datasets due to the different dimensionality of the variables and will be analyzed further in subsequent experiments.

For other deep learning baselines, we use the parameter settings recommended by their authors and run the code provided by them to obtain the comparative experimental outputs.

5.4. Empirical results

The final outputs of the multivariate time-series forecasting task using various methods are summarized in Tables 2 and 3, where the forecasting horizon is set as $\{3, 6, 9, 12\}$. From the results, we observe that the result has a certain degree of decrease as the horizon increases, indicating that the larger horizons are, the more difficult it is to make accurate predictions.

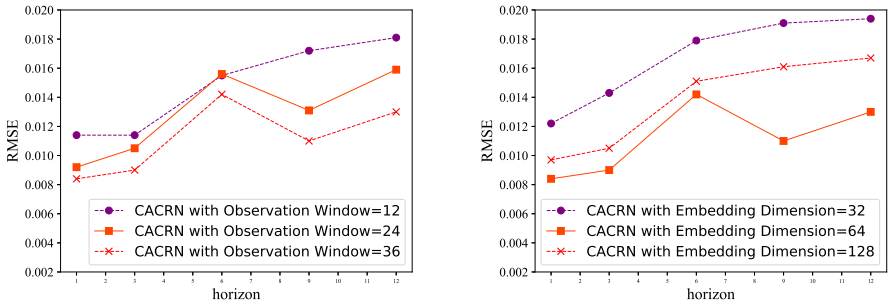
Obviously, our proposed CACRN outperforms the state-of-the-art methods. It is evident that CACRN has brought about 25% and 10.5% RMSE improvement respectively y in Traffic and Solar-Energy ($0.028 \rightarrow 0.021, 0.019 \rightarrow 0.017$), and 60%, 41.7%, 41.2% and 21.4% MAE improvement respectively in PeMSD7, Air Quality, Wiki^{small} and Electricity compared with the best baselines ($0.015 \rightarrow 0.006, 0.012 \rightarrow 0.007, 0.017 \rightarrow 0.010, 0.014 \rightarrow 0.011$).

Specifically, we observed that the VAR method performed poorly on the Solar-Energy dataset due to its irregularity. The dataset contains electricity consumption information every 5 min for one year, with most of the data being 0 during the night period. Thus, the traditional VAR model finds it difficult to capture such a data pattern. Furthermore, its feature learning ability is much inferior to that of deep learning methods, resulting in poor empirical results. Other typical deep learning models depend on RNN, CNN, LSTM, and transformers giving less consideration to inter-series dependencies, resulting in poor prediction performance for datasets with high dimensions. Among these methods, we found that the DeepGLO model focuses on both local and global temporal dependencies, making it more effective than the others. Therefore, inspired by DeepGLO, CACRN's convolutional recurrent neural network captures the short-term and long-term information of temporal patterns through the joint construction of convolution and recurrent components, thereby achieving better prediction results.

Furthermore, CACRN's superiority over other baseline models can be attributed to its ability to capture inter-series dependencies. The PeMSD7 and Air Quality datasets show significant improvements in metrics as their variables have stronger dependencies. For instance, PeMSD7 records traffic flow data of expressways, where connected expressways affect each other's traffic flow data, but expressways far away from each other are not related. On the other hand, Air Quality contains multiple measurement indexes with positive, negative, or uncorrelated relationships among them. Therefore, CACRN achieves better results by accounting for variable dependencies. The effectiveness of MTGNN and CATN is also due to their consideration of variable relationships. However, MTGNN's graph structure may struggle to learn out-of-order graphs, while CATN's tree structure requires determining the number of layers to get results, which can be challenging and less accurate based on prior experience. By contrast, CACRN utilizes Variable Fuzzy Clustering to divide variables without over-reliance on k-values, followed by Variable Interaction Representation to capture interaction information between related variables within the same cluster. Finally, the Cluster-aware Attention Mechanism allows CACRN to obtain the different contributions of irrelevant clusters and capture inter-cluster dependencies. Consequently, CACRN effectively models inter-series dependencies by capturing intra-cluster and inter-cluster dependencies between series. We will discuss the contribution of different modules to the model in subsequent ablation studies.

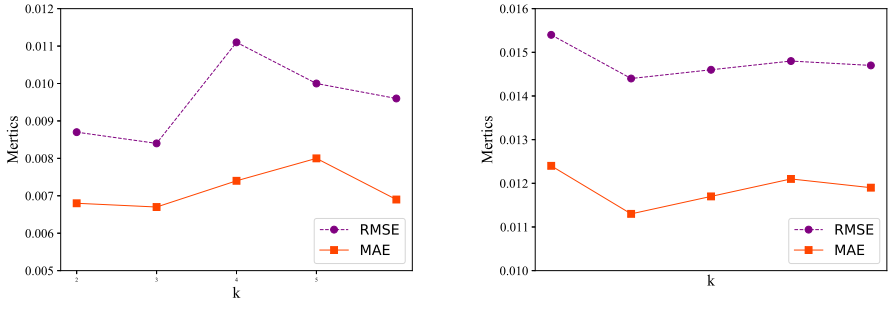
Table 3
Metrics results (RMSE, MAE) of all methods on the datasets of PeMSD7, Solar-Energy, Air Quality. In each case, the best performance is displayed in **boldface**. (1) each row contains the two-metric findings of a given approach; (2) each column evaluates the outputs of all approaches on a specific dataset with different horizons.

Models	Metrics	PeMSD7				Solar-Energy				Air Quality			
		3	6	9	12	3	6	9	12	3	6	9	12
VAR	RMSE	0.057	0.077	0.093	0.109	0.864	1.432	3.712	13.26	0.236	0.337	0.698	0.943
	MAE	0.036	0.048	0.059	0.069	0.026	0.439	1.712	3.244	0.098	0.211	0.568	0.992
DARNN	RMSE	0.251	0.332	0.383	0.421	0.098	0.144	0.196	0.247	0.647	0.831	0.898	0.911
	MAE	0.427	0.556	0.624	0.673	0.169	0.239	0.308	0.262	0.411	0.536	0.608	0.585
LSTNet	RMSE	0.131	0.056	0.072	0.083	0.091	0.051	0.059	0.068	0.137	0.146	0.154	0.154
	MAE	0.032	0.041	0.047	0.052	0.031	0.033	0.041	0.065	0.081	0.091	0.101	0.103
TPA-LSTM	RMSE	0.065	0.529	0.571	0.603	0.186	0.264	0.278	0.298	0.218	0.249	0.375	0.461
	MAE	0.038	0.312	0.338	0.354	0.093	0.143	0.145	0.161	0.139	0.164	0.252	0.313
MTGNN	RMSE	0.059	0.077	0.091	0.099	0.022	0.033	0.041	0.049	0.108	0.131	0.133	0.133
	MAE	0.029	0.043	0.051	0.058	0.011	0.021	0.022	0.027	0.051	0.072	0.073	0.077
DeepGLO	RMSE	0.066	0.075	0.051	0.065	0.017	0.019	0.019	0.022	0.024	0.026	0.025	0.028
	MAE	0.038	0.045	0.023	0.036	0.053	0.017	0.055	0.018	0.018	0.016	0.017	0.019
Informer	RMSE	0.012	0.013	0.015	0.013	0.024	0.025	0.026	0.028	0.022	0.024	0.024	0.024
	MAE	0.009	0.009	0.013	0.015	0.014	0.015	0.017	0.019	0.014	0.015	0.014	0.014
TS2Vec	RMSE	0.053	0.057	0.061	0.063	0.026	0.029	0.032	0.035	0.059	0.070	0.074	0.078
	MAE	0.036	0.038	0.040	0.041	0.018	0.021	0.023	0.025	0.038	0.046	0.049	0.052
CATN	RMSE	0.037	0.036	0.036	0.039	0.026	0.026	0.026	0.027	0.017	0.017	0.019	0.018
	MAE	0.014	0.014	0.014	0.016	0.018	0.018	0.019	0.018	0.012	0.012	0.014	0.014
CACRN (ours)	RMSE	**0.011**	**0.008**	**0.008**	**0.009**	**0.016**	**0.017**	**0.018**	**0.021**	**0.008**	**0.011**	**0.014**	**0.017**
	MAE	**0.007**	**0.005**	**0.005**	**0.006**	**0.013**	**0.014**	**0.016**	**0.017**	**0.007**	**0.009**	**0.011**	**0.013**



(a) Observation Window (b) Embedding Dimension

Fig. 5. The parameter sensitivity analysis of CACRN on Air Quality dataset.



(a) Air Quality (b) Electricity

Fig. 6. The parameter sensitivity of Variable Fuzzy Clustering on different datasets.

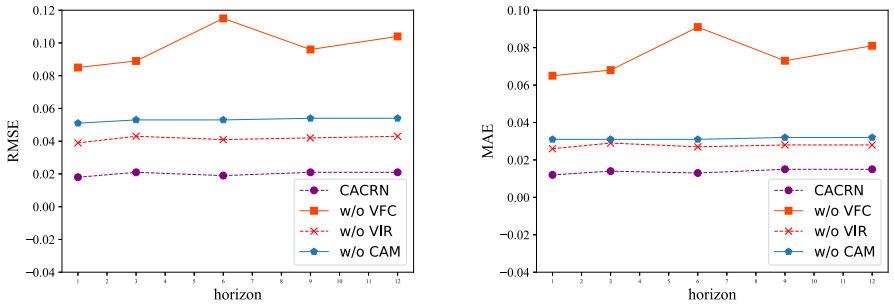
5.5. Parameter sensitivity

In order to demonstrate the impact of CACRN parameters for forecasting, we perform a parameter analysis on Air Quality in terms of window length, embedding dimension, and cluster number in Variable Fuzzy Clustering.

Observation Window: As depicted in Fig. 5(a), it is observed that the RMSE value decreases as the observation window length increases. This indicates that a longer observation window enables the convolutional recurrent neural network to more efficiently gather short- and long-term temporal patterns' information. Consequently, we believe that increasing the observation window length enhances the forecasting results' effectiveness. With more extended historical data, more information is available, which enables us to identify inter-series relationships and short- and long-term temporal intra-series trends more accurately.

Table 4
Metrics results (RMSE, MAE) of CACRN and three variants of the model on the Traffic dataset. The best performance is displayed in **boldface** in each case.

Models	CACRN				w/o VFC				w/o VIR				w/o CAM			
	3	6	9	12	3	6	9	12	3	6	9	12	3	6	9	12
RMSE	**0.021**	**0.019**	**0.021**	**0.021**	0.089	0.115	0.096	0.104	0.043	0.041	0.042	0.043	0.053	0.053	0.054	0.054
MAE	**0.014**	**0.013**	**0.015**	**0.015**	0.068	0.091	0.073	0.081	0.029	0.027	0.028	0.028	0.031	0.031	0.032	0.032



(a) RMSE (b) MAE

Fig. 7. The ablation study of CACRN on Traffic dataset.

Embedding Dimension: As illustrated in Fig. 5(b), the RMSE value is found to be highest when the dimension equals 32 in all three experimental groups. Furthermore, the forecasting performance of dimension 64 surpasses that of dimension 128. A larger dimension can cause an increase in the computational burden on the model and can make information aggregation more challenging. Our proposed model CACRN achieves the highest performance when the embedding dimension is equal to 64.

Variable Fuzzy Clustering: As shown in Fig. 6, the value of k in Variable Fuzzy Clustering has a significant impact on the RMSE and MAE results. However, the optimal value of k varies among different datasets. For instance, the smallest RMSE value for Air Quality is achieved with $k = 3$, which outperforms other values. On the other hand, for Electricity, the model's effectiveness is significantly better with $k = 11$ compared to other values. A too-small k value cannot effectively separate variables, leading to insufficient reduction of mutual interference between variables with low correlation. A too-large k value makes the number of parallelization of convolutional recurrent neural networks too large, which can increase the difficulty of information communication and aggregation, and result in a more complex model, leading to a decline in the forecasting results. During the experiments, the range of k values is calculated using the VFC method, and the validation experiments are completed within the appropriate range. The best k-value for each dataset is obtained based on the validation results. The obtained experimental results are collected and analyzed when selecting the best k-value.

5.6. Ablation study

We created three model variations and investigate their performance on Traffic to further assess the effect of the various components in CACRN. The three variants of CACRN are shown below:

w/o VFC. CACRN without the Variable Fuzzy Clustering (VFC). We feed all data into the model without separating uncorrelated variables from each other.

w/o VIR. CACRN without the Variable Interaction Representation (VIR). For the variables in each cluster, we directly combine and feed them into the next Convolutional Recurrent Neural Network module.

w/o CAM. CACRN without the Cluster-aware Attention Mechanism (CAM). We cancel the Cluster-aware Attention Mechanism and input the intermediate process outputs directly into the fully connected layers (FC).

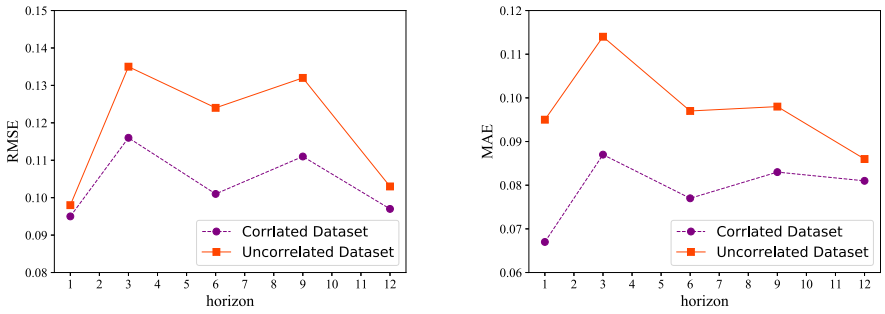
Table 4 and Fig. 7 show the final outputs of three model variations in ablation experiments, from which we can assess the effectiveness of different components proposed in CACRN. Among them, **w/o VFC** significantly increased the value of RMSE/MAE (76.4%, 79.4%). The Variable Fuzzy Clustering method can separate irrelevant variables from each other according to the change of time. This is particularly important for capturing the interactive information in VIR, and the intra-cluster and inter-cluster dependencies in CAM. Due to the **w/o VIR** did not capture the interaction information between relevant variables, the value of RMSE/MAE also increased (51.2%, 51.7%). From the baseline models, we can learn that it is not enough to merely separate the variables. Relevant variables still need to capture their interactive information. So CACRN uses VIR to cross-fuse them and obtain a better representation, which can improve the accuracy of forecasting. **w/o CAM** does not capture the inter-series and intra-series dependencies, the value of RMSE/MAE is improved (60.4%, 54.8%). According to the results, if all variables are cross-fused directly, the forecasting accuracy is greatly reduced. This is because w/o CAM cannot consider the relationship between variables, so it cannot analyze the dependency between clusters. Therefore, it is necessary for CAM to both capture the inter-cluster and the intra-cluster dependencies.

5.7. Inter-series dependency analysis

In addition, we investigate the influence of the inter-series dependencies in CACRN. Specifically, we experiment with correlated and uncorrelated data independently to investigate whether the dependencies between variables affect forecasting results. We complete this experiment on the Air Quality dataset.

As the variables are divided into clusters, the dataset can be split into two sub-datasets. The first sub-dataset, named the correlated dataset, includes four variables from the same cluster in the VFC results. The second sub-dataset, named the uncorrelated dataset, contains five variables, including four variables from the first sub-dataset and a variable from a different cluster. These two sub-datasets are used in a CACRN model variant that only has the prediction function. Table 5 and Fig. 8 display the results.

The results clearly indicate that the addition of uncorrelated variables to the second sub-dataset significantly decreases the model performance. Upon further analysis, it was found that the four variables included in the correlated dataset were related to each other, such as positive and negative correlations. These variables, namely benzene concentration, titania, tin oxide, and indium oxide, have a closer



(a) RMSE (b) MAE

Fig. 8. The inter-Series dependency analysis on Air Quality dataset.

Table 5
Metrics results (RMSE, MAE) of the CACRN model variant on two sub-datasets.

Datasets	Correlated sub-dataset				Uncorrelated sub-dataset			
	3	6	9	12	3	6	9	12
RMSE	0.116	0.101	0.111	0.097	0.135	0.124	0.132	0.103
MAE	0.087	0.077	0.083	0.081	0.114	0.097	0.098	0.081

relationship from an environmental chemistry perspective. Therefore, by inputting these four correlated variables simultaneously into the forecasting network, the model achieves excellent results. However, the variable added to the uncorrelated dataset is the average relative humidity information, which has a low correlation with other variables. As a result, the common input of these five variables to the forecasting leads to poor results.

The results of this case study show that inter-series dependencies have a significant influence on the prediction performance of multivariate time series. The Cluster-aware Variable Representation Module in CACRN effectively captures the dependencies between variables, which greatly impacts the subsequent forecasting. It is unreasonable to use all variables in the dataset for forecasting directly. Instead, by separating uncorrelated variables and cross-fusing correlated variables, we can achieve better forecasting performance. This highlights the importance of considering the relationships between variables in multivariate time series analysis.

6. Conclusion

In this study, we investigated variable dependencies in multivariate time series and proposed CACRN, a model for multi-step forecasting. By using Variable Fuzzy clustering and Variable Interaction Representation, we can separate time-series variables that have low correlation with each other and capture the dependencies between them in a cluster-wise manner. We introduced a Convolutional Recurrent Neural Network to capture both short- and long-term temporal patterns and a Cluster-aware Attention Mechanism to capture inter-cluster and intra-cluster dependencies. Our CACRN achieved superior performance on six real-world datasets, and we conducted further analysis and empirical evidence to confirm the effectiveness of the architecture. Importantly, CACRN is suitable for real-world data in various fields, particularly those with clear variable dependencies.

CRediT authorship contribution statement

Simeng Bai: Writing – original draft, Conceptualization, Methodology. **Qi Zhang:** Conceptualization, Writing – review & editing. **Hui He:** Investigation, Data curation. **Liang Hu:** Visualization, Writing – review & editing. **Shoujin Wang:** Validation, Writing – review & editing. **Zhendong Niu:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The work is supported by the National Natural Science Foundation of China (62272048) and the National Key Research and Development Program of China (2019YFB1406302).

References

[1] Y. Liu, C. Wu, J. Wen, X. Xiao, Z. Chen, A grey convolutional neural network model for traffic flow prediction under traffic accidents, Neurocomputing 500 (2022) 761–775.
[2] P. Chatigny, J. Patenaude, S. Wang, Spatiotemporal adaptive neural network for long-term forecasting of financial time series, Internat. J. Approx. Reason. 132 (2021) 70–85.
[3] S. Du, T. Li, Y. Yang, S. Horng, Deep air quality forecasting using hybrid deep learning framework, IEEE Trans. Knowl. Data Eng. 33 (6) (2021) 2412–2424.
[4] N. Jin, F. Yang, Y. Mo, Y. Zeng, X. Zhou, K. Yan, X. Ma, Highly accurate energy consumption forecasting model based on parallel LSTM neural networks, Adv. Eng. Inform. 51 (2022) 101442.
[5] M. Krishnan, R. Sakthivel, R. Vishnuvarthan, Neural network based optimization approach for energy demand prediction in smart grid, Neurocomputing 273 (2018) 199–208.
[6] G. Lai, W. Chang, Y. Yang, H. Liu, Modeling long- and short-term temporal patterns with deep neural networks, in: K. Collins-Thompson, Q. Mei, B.D. Davison, Y. Liu, E. Yilmaz (Eds.), The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, SIGIR 2018, Ann Arbor, MI, USA, July 08-12, 2018, ACM, 2018, pp. 95–104, http://dx.doi.org/10.1145/3209978.3210006.
[7] S. Shih, F. Sun, H. Lee, Temporal pattern attention for multivariate time series forecasting, Mach. Learn. 108 (8–9) (2019) 1421–1441, http://dx.doi.org/10.1007/s10994-019-05815-0.
[8] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, in: AAAI, AAAI Press, 2021, pp. 11106–11115.
[9] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, C. Zhang, Connecting the dots: Multivariate time series forecasting with graph neural networks, in: R. Gupta, Y. Liu, J. Tang, B.A. Prakash (Eds.), KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23-27, 2020, ACM, 2020, pp. 753–763, http://dx.doi.org/10.1145/3394486.3403118.

[10] D. Cao, Y. Wang, J. Duan, C. Zhang, X. Zhu, C. Huang, Y. Tong, B. Xu, J. Bai, J. Tong, Q. Zhang, Spectral temporal graph neural network for multivariate time-series forecasting, in: NeurIPS, 2020.
[11] J.K. Tarus, Z. Niu, G. Mustafa, Knowledge-based recommendation: a review of ontology-based recommender systems for e-learning, Artif. Intell. Rev. 50 (1) (2018) 21–48.
[12] B. Yu, H. Yin, Z. Zhu, Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting, in: IJCAI, 2018, pp. 3634–3640.
[13] G. He, H. Wang, S. Liu, B. Zhang, CSMVC: A multiview method for multivariate time-series clustering, IEEE Trans. Cybern. (2021) 1–13.
[14] Q. Ma, S. Li, W. Zhuang, S. Li, J. Wang, D. Zeng, Self-supervised time series clustering with model-based dynamics, IEEE Trans. Neural Netw. Learn. Syst. 32 (9) (2021) 3942–3955.
[15] Q. Ma, C. Chen, S. Li, G.W. Cottrell, Learning representations for incomplete time series clustering, in: AAAI, 2021, pp. 8837–8846.
[16] Q. Zhang, L. Cao, C. Shi, Z. Niu, Neural time-aware sequential recommendation by jointly modeling preference dynamics and explicit feature couplings, IEEE Trans. Neural Netw. Learn. Syst. (2021) 1–13.
[17] P. Qiu, Z. Niu, TCIC_FS: Total correlation information coefficient-based feature selection method for high-dimensional data, Knowl.-Based Syst. 231 (2021) 107418.
[18] Ü.Ç. Büyüksahin, S. Ertekin, Improving forecasting accuracy of time series data using a new ARIMA-ANN hybrid method and empirical mode decomposition, Neurocomputing 361 (2019) 151–163.
[19] S. Chadsuthi, S. Iamsirithaworn, W. Triampo, C. Modchang, Modeling seasonal influenza transmission and its association with climate factors in thailand using time-series and ARIMAX analyses, Comput. Math. Methods Med. 2015 (2015) 436495:1–436495:8.
[20] J.M. Corrêa, A.C. Neto, L.A.T. Junior, E.M. Careño, Á.E. Faria, Time series forecasting with the WARIMAX-GARCH method, Neurocomputing 216 (2016) 805–815.
[21] T. Hsieh, Y. Sun, X. Tang, S. Wang, V.G. Honavar, SrVARM: State regularized vector autoregressive model for joint learning of hidden state transitions and state-dependent inter-variable dependencies from multi-variate time series, in: WWW, ACM / IW3C2, 2021, pp. 2270–2280.
[22] P. Taveeapiradeecharoen, K. Chamnongthai, N. Aunsri, Bayesian compressed vector autoregression for financial time-series analysis and forecasting, IEEE Access 7 (2019) 16777–16786.
[23] F.A.A. Bashir, H. Wei, Handling missing data in multivariate time series using a vector autoregressive model-imputation (VAR-IM) algorithm, Neurocomputing 276 (2018) 23–30.
[24] A.M. Tripathi, Enhancing multivariate time series classification using LSTM and evidence feed forward HMM, in: 2020 International Joint Conference on Neural Networks, IJCNN 2020, Glasgow, United Kingdom, July 19-24, 2020, IEEE, 2020, pp. 1–7, http://dx.doi.org/10.1109/IJCNN48605.2020.9207636.
[25] J. Hu, W. Zheng, Transformation-gated LSTM: efficient capture of short-term mutation dependencies for multivariate time series prediction tasks, in: International Joint Conference on Neural Networks, IJCNN 2019 Budapest, Hungary, July 14-19, 2019, IEEE, 2019, pp. 1–8, http://dx.doi.org/10.1109/IJCNN.2019.8852073.
[26] H. He, Q. Zhang, S. Bai, K. Yi, Z. Niu, CATN: Cross attentive tree-aware network for multivariate time series forecasting, in: AAAI, Vol. 36, 2022, pp. 4030–4038.
[27] Q. Tan, M. Ye, B. Yang, S. Liu, A.J. Ma, T.C. Yip, G.L. Wong, P.C. Yuen, DATA-GRU: dual-attention time-aware gated recurrent unit for irregular multivariate time series, in: AAAI, AAAI Press, 2020, pp. 930–937.
[28] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting, in: NeurIPS, 2021, pp. 22419–22430.
[29] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, C. Eickhoff, A transformer-based framework for multivariate time series representation learning, in: KDD, 2021, pp. 2114–2124.
[30] K.D. Hartomo, Y. Nataliani, A new model for learning-based forecasting procedure by combining k-means clustering and time series forecasting algorithms, PeerJ Comput. Sci. 7 (2021) e534, http://dx.doi.org/10.7717/peerj-cs.534.
[31] J. Paparrizos, L. Gravano, Fast and accurate time-series clustering, ACM Trans. Database Syst. 42 (2) (2017) 8:1–8:49.
[32] Q. Ma, J. Zheng, S. Li, G.W. Cottrell, Learning representations for time series clustering, in: NeurIPS, 2019, pp. 3776–3786.
[33] J. Yang, J. Leskovec, Patterns of temporal variation in online media, in: WSDM, 2011, pp. 177–186.
[34] J. Paparrizos, L. Gravano, k-shape: Efficient and accurate clustering of time series, in: SIGMOD, 2015, pp. 1855–1870.
[35] H. He, Q. Zhang, K. Yi, K. Shi, Z. Niu, L. Cao, Distributional drift adaptation with temporal conditional variational autoencoder for multivariate time series forecasting, 2022, CoRR abs/2209.00654.
[36] R. Cerqueti, P. D'Urso, L.D. Giovanni, M. Giacalone, R. Mattera, Weighted score-driven fuzzy clustering of time series with a financial application, Expert Syst. Appl. 198 (2022) 116752.
[37] W. Alahamade, I. Lake, C.E. Reeves, B. de la Iglesia, A multi-variate time series clustering approach based on intermediate fusion: A case study in air pollution data imputation, Neurocomputing 490 (2022) 229–245.
[38] Y. Liu, S. Ding, W. Jia, A novel prediction method of complex univariate time series based on k-means clustering, Soft Comput. 24 (21) (2020) 16425–16437.
[39] E. Fu, Y. Zhang, F. Yang, S. Wang, Temporal self-attention-based conv-LSTM network for multivariate time series prediction, Neurocomputing 501 (2022) 162–173.
[40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: NIPS, 2017, pp. 5998–6008.
[41] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: Y. Bengio, Y. LeCun (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings, 2015.
[42] R. Sen, H. Yu, I.S. Dhillon, Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting, in: NeurIPS, 2019, pp. 4838–4847.
[43] R. Jankovic, I. Mihajlovic, A. Amelio, Time series vector autoregression prediction of the ecological footprint based on energy parameters, 2019, CoRR abs/1910.11800.
[44] Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, G.W. Cottrell, A dual-stage attention-based recurrent neural network for time series prediction, in: C. Sierra (Ed.), Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017, ijcai.org, 2017, pp. 2627–2633, http://dx.doi.org/10.24963/ijcai.2017/366.
[45] Z. Yue, Y. Wang, J. Duan, T. Yang, C. Huang, Y. Tong, B. Xu, TS2Vec: Towards universal representation of time series, in: AAAI, AAAI Press, 2022, pp. 8980–8987.
[46] N. Srivastava, G.E. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, J. Mach. Learn. Res. 15 (1) (2014) 1929–1958.



Simeng Bai received the M.E. degree from Beijing Institute of Technology, Beijing, China in 2022. She is currently pursuing the Ph.D. degree at School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China. Her current research interests focus on multivariate time-series analysis and data privacy.



Qi Zhang received his Ph.D. degrees from Beijing Institute of Technology China under the dual Ph.D. program of Beijing Institute of Technology and University of Technology Sydney Australia. He is currently an AI Scientist in DeepBlue Academy of Sciences. His current research interests focus on Collaborative Filtering, learning to hash and sequential recommendation and multivariate time-series analysis.



Hui He received the M.E. degree from University of Shanghai for Science and Technology, Shanghai, China in 2020. She is currently pursuing the Ph.D. degree at Institute of Engineering Medicine, Beijing Institute of Technology, Beijing, China. Her current research interests focus on multivariate time-series analysis and knowledge services.



Liang Hu received dual Ph.D. degrees from Shanghai Jiao Tong University, China and University of Technology Sydney, Australia. He is currently a Distinguished Research Fellow in Tongji University and Chief AI Scientist in DeepBlue Academy of Sciences. His research interests include recommender systems, machine learning, data science and general intelligence.



Shoujin Wang received a Ph.D. in Analytics from University of Technology Sydney in 2019. He is currently a research fellow of data science and artificial intelligence with School of Computing Technologies, RMIT University, Melbourne, Australia. He is also with the School of Computing, Macquarie University, Sydney, Australia. His primary research interests include data mining, machine learning, recommender systems, fake news detection and mitigation.



Zhendong Niu received a Ph.D. degree in computer science from the Beijing Institute of Technology, Beijing, China, in 1995. His current research interests include informational retrieval, software architecture, digital libraries, and Web-based learning techniques. He is a recipient of the IBM Faculty Innovation Award in 2005 and the New Century Excellent Talents, University of Ministry of Education, China, in 2006.